Résoudre Meowdoku par SAT

Rapport global de synthèse

Objectif. Construire en Python 3.12 une chaîne complète de résolution de Meowdoku, depuis la modélisation logique jusqu'à un solveur optimisé, puis évaluer sa correction et ses performances. Le programme utilise uniquement la bibliothèque standard de Python.

Grille ⟶ CNF / DIMACS ⟶ solveur SAT ⟶ placement vérifié

1. Modélisation et génération des grilles

Une variable $x_{i,j}$ représente la présence d'un chat dans la case (i, j) d'une grille de N lignes, M colonnes et K régions. L'encodage comporte NM variables et deux formes de clauses :

$$\underbrace{\bigvee_{u \in R} x_u}_{\text{au moins un chat dans chaque région } R} \qquad \text{et} \qquad \underbrace{\neg x_u \vee \neg x_v}_{\text{deux cases incompatibles } u,v} .$$

Les paires incompatibles imposent au plus un chat par ligne, colonne et région, ainsi que l'absence de chats voisins, diagonales comprises. Ainsi, la formule est satisfiable **si et seulement si** la grille admet un placement valide. Les clauses identiques sont regroupées et la numérotation DIMACS est $\mathrm{id}(i, j) = (i - 1)M + j$.

Le générateur se limite au cas autorisé $N = M = K = n$. Il construit d'abord un placement valide par retour arrière, puis fait grandir une région connexe autour de chaque chat. Une solution existe donc par construction, mais son unicité n'est pas garantie. Les tailles $n = 2$ et $n = 3$, impossibles, sont rejetées.

2. Lecture DIMACS et DPLL de référence

Les clauses sont représentées par des ensembles immuables de littéraux entiers. Les deux versions demandées de DPLL sont implémentées : la première décide SAT/UNSAT ; la seconde fournit aussi une valuation en cas de satisfiabilité.

Le solveur détecte une clause vide ou une formule vide, applique la propagation unitaire et la règle du littéral pur, puis choisit aléatoirement une variable et sa valeur. En cas d'échec, il explore la valeur opposée. Cette version sert de référence ; la recopie de la formule à chaque simplification et la taille de l'arbre de recherche limitent sa vitesse.

3. Résolution et restitution d'une grille

`meowdoku.py` convertit la grille en DIMACS, appelle le DPLL de référence et transforme les variables vraies en coordonnées de chats. Il vérifie à la fois les clauses SAT et, indépendamment, les règles du jeu. Le fichier résultat reprend l'entrée, les coordonnées par région et une grille marquée de `*`. Pour une grille UNSAT, aucun résultat n'est écrit. Le moteur optimisé de la question 6 est accessible séparément par `sat.py`.

4. Heuristiques de branchement

L'algorithme DPLL est conservé ; seule sa fonction de choix devient paramétrable. Cette séparation permet de comparer les heuristiques avec les mêmes règles de propagation et de construction du modèle. Les 16 variantes couvrent les critères suivants :

Famille	Critère et justification
H0 / H1	Hasard / première variable libre : références peu coûteuses.
H2 / H3	Variable / littéral le plus fréquent : agir sur de nombreuses clauses.
H4 / H5	Fréquences dans les clauses les plus courtes : provoquer des unités et révéler plus tôt les contradictions.
H6 / H7	Variable / littéral de meilleur poids de Jeroslow–Wang : $J(l) = \sum_{C \ni l} 2^{-\lvert C \rvert}$ privilégie les clauses courtes sans ignorer les autres.
H8	Priorité aux littéraux positifs, départagés par leur poids : sur Meowdoku, essayer de placer un chat.

H2 à H8 ont une variante **statique** (scores initiaux) et une variante **dynamique** (scores recalculés sur la formule résiduelle). Le statique économise du calcul, mais son classement peut devenir moins pertinent. Le dynamique tient compte des simplifications, au prix d'un parcours supplémentaire de la formule à chaque décision.

H8 répond à la structure de Meowdoku : les nombreuses exclusions binaires négatives dominent les fréquences, tandis que les clauses positives de régions sont peu nombreuses. Favoriser un littéral positif déclenche les exclusions associées à un placement. Ce choix est une hypothèse à tester, pas une garantie de supériorité sur toute CNF.

5. Analyse des performances

5.1 Protocole et lecture des indicateurs

La campagne porte sur **173 instances et 2 768 résolutions** : 50 instances 3-SAT SAT, 50 UNSAT, 23 instances structurées (17 SAT, 6 UNSAT), 20 Meowdoku SAT de tailles 8 à 16 et 30 grilles à régions aléatoires (7 SAT, 23 UNSAT). La limite est de 60 s réelles par résolution ; les processus mesurent chacun leur temps CPU.

Les décisions et retours arrière décrivent l'arbre exploré ; les propagations et littéraux purs renseignent sur les simplifications. Le temps CPU mesure leur coût global. Le nombre de résolutions terminées et le **PAR-2**, moyenne des temps CPU où un dépassement compte pour 120 s, empêchent d'ignorer les instances difficiles.

Les compteurs et gains ci-dessous sont comparés sur les instances résolues par *toutes* les heuristiques du groupe, pour ne pas mélanger travail achevé et compteurs interrompus. Pour les temps, le gain est la moyenne géométrique des rapports instance par instance, $G = \exp\big(\frac{1}{m}\sum_i \ln(t_{\mathrm{H0},i}/t_{\mathrm{H},i})\big)$. $G > 1$ indique une accélération. Ce sous-ensemble doit être lu avec les taux de résolution : il exclut certaines instances difficiles.

Gain CPU sur H0	3-SAT SAT (50)	3-SAT UNSAT (50)	Meowdoku SAT (19)
H2s	6,8	7,9	0,9
H4s	7,2	7,9	0,8
H4d	**11,0**	**14,6**	0,8
H8s	8,2	5,2	**3,3**
H8d	7,2	6,3	2,7

Effectifs communs entre parenthèses. Les cinq variantes résolvent les 20 grilles Meowdoku SAT.

5.2 Résultats conformes aux intuitions et nuances

Sur le 3-SAT, privilégier les clauses courtes est efficace. H4d fait passer la médiane des décisions de 249 à 17 sur les SAT et d'environ 568 à 40 sur les UNSAT. Ses gains CPU atteignent respectivement 11,0 et 14,6. Les clauses courtes deviennent rapidement unitaires ou contradictoires : leur priorité réduit l'exploration. Le gain plus marqué en UNSAT est cohérent avec l'obligation de réfuter toutes les branches encore possibles ; en SAT, une branche favorable peut suffire.

Sur les instances structurées, il faut aussi regarder les délais. H4d, H5d et H8d résolvent les 17 SAT, contre 15 pour H0. Aucune variante ne termine `public-03` en 60 s. Sur les cinq UNSAT résolues par toutes, H4s obtient un gain CPU de 25,5 : ce résultat ne doit pas masquer la sixième instance non résolue.

Sur Meowdoku, la polarité est déterminante, mais les décisions ne suffisent pas à mesurer la difficulté. Sur les 19 grilles SAT communes, H8s a une médiane de 10 décisions et 3 retours arrière, contre 54 et 43 pour H0. À l'inverse, H2d atteint 115 décisions mais zéro retour arrière en médiane : exclure les cases une à une peut être long sans conduire à beaucoup d'impasses. H8s gagne un facteur 3,3 en temps CPU ; sur les quatre grilles 16×16, sa médiane est de 0,068 s contre 0,530 s pour H0. Ce rapport de médianes est distinct du gain géométrique global.

Les 23 grilles aléatoires UNSAT sont souvent réfutées après très peu de décisions. Elles testent la détection de contradictions, mais ne suffisent pas à représenter les Meowdoku UNSAT difficiles.

5.3 Le recalcul dynamique vaut-il son coût ?

Famille	Décisions s/d	Coût d'un choix d/s	Temps CPU s/d	Résolues s / d
H2	1,28	6,3	0,87	169 / 170
H4	1,74	5,0	1,31	171 / 172
H6	1,79	7,0	1,17	171 / 171
H8	1,34	6,4	0,97	170 / 172

Moyennes géométriques sur les instances résolues par les deux variantes ; pour les décisions, le rapport est $(D_s + 1)/(D_d + 1)$. Les nombres résolus portent sur les 173 instances. Le coût d'un choix utilise la durée cumulée de l'heuristique divisée par ses décisions.

Le dynamique fait moins de décisions *en moyenne sur ces paires*, sans garantie instance par instance. Pour H4, le rapport CPU 1,31 correspond à environ 24 % de durée en moins ; pour H6, 1,17 correspond à environ 15 %. L'actualisation des petites clauses compense ici son coût. Pour H2, le rapport 0,87 traduit au contraire environ 15 % de durée en plus : l'amélioration de l'arbre ne rembourse pas les recalculs.

H8 illustre un autre compromis : le dynamique est légèrement plus lent sur les paires communes, mais résout deux instances supplémentaires. Sur Meowdoku SAT, H8s consacre environ 5 % de sa durée à l'heuristique, contre 28 % pour H8d. Le choix final doit donc associer **qualité du branchement, coût du calcul et taux de résolution**. Ces observations conduisent à conserver une priorité aux placements, tout en réduisant les parcours de la formule dans le moteur optimisé.

6. Solveur optimisé : conception et résultats

6.1 Réduire le coût d'une étape de recherche

Le moteur `optimized.py` conserve DPLL, mais remplace les copies de formule par des structures modifiées en place :

— Une clause binaire $(a \vee b)$ devient les implications $\neg a \Rightarrow b$ et $\neg b \Rightarrow a$. Placer un chat propage directement les exclusions des cases incompatibles.

— Les clauses longues ont deux littéraux surveillés. Seules les clauses dont une surveillance devient fausse doivent être examinées pour déplacer la surveillance, propager une unité ou détecter un conflit.

— Une pile mémorise les affectations ; le retour arrière annule celles du niveau abandonné. La recherche itérative évite une pile d'appels récursifs profonde.

Les tautologies et doublons sont éliminés au départ. La recherche de littéraux purs est supprimée pour éviter un parcours global ; cette règle n'est pas indispensable à la complétude de DPLL.

Pour une CNF composée uniquement de clauses positives et d'exclusions binaires négatives, le moteur choisit dans la clause positive non satisfaite qui a le moins de candidats libres. Les scores positifs initiaux de Jeroslow–Wang départagent ses candidats. Ce choix adapte H8 à Meowdoku : il actualise la contrainte la plus urgente en ne parcourant que les clauses positives. Ce n'est pas exactement H8d pour des clauses positives qui se recouvrent. Les autres CNF utilisent H4d.

Correction. Toute décision garde une branche opposée. Les implications et surveillances préservent les clauses d'origine. Au retour arrière, aucune affectation ne devient fausse : elle devient libre, ce qui permet de conserver les surveillances. Dans la classe de placement, une fois les clauses positives satisfaites, compléter à faux les variables restantes préserve les exclusions déjà propagées. Le moteur reste général ; sa structure n'impose pas qu'une solution existe.

6.2 Une évaluation distincte, proche de l'interface finale

Le banc de mesure lance `sat.py` avec son modèle dans un nouveau processus pour chaque essai. Les 91 instances comprennent les 41 fichiers publics et les 50 grilles Meowdoku ; les deux modes sont lancés séquentiellement avec un délai de 60 s, soit 182 essais. Les statuts attendus proviennent des fichiers publics, de la construction des grilles SAT et de l'accord observé en Q5 pour les régions aléatoires.

Indicateur	DPLL initial	Optimisé
Instances correctement résolues	89/91	**90/91**
Benchmarks publics résolus	28/30	29/30
Tests publics d'interface et cas limites	11/11	11/11
Grilles Meowdoku résolues	50/50	50/50
Temps complet sur les 50 grilles Meowdoku	3,01 s	1,62 s
PAR-2 sur les 91 instances (temps complet)	3,439 s	1,421 s

Le temps complet inclut Python, les imports, la lecture, la résolution et la sortie du modèle ; sa vérification intervient ensuite, hors mesure. Le PAR-2 compte chaque dépassement pour 120 s. Le classement prévu privilégie d'abord le nombre de réponses correctes, puis leur temps total. Sur ce corpus, l'optimisation apporte donc un bénéfice de robustesse en plus du gain de vitesse.

6.3 Interpréter les gains et les contre-exemples

Sur les 50 grilles Meowdoku, le gain moyen géométrique est de **4,2 en temps CPU du solveur** et de **1,5 en temps complet**. Les coûts fixes du lancement et de la lecture limitent le gain visible sur les petites instances. Sur les 89 instances résolues par les deux modes, le gain géométrique en temps complet est de 1,81.

Instance publique	DPLL (s)	Optimisé (s)	Observation
03	> 60	> 60	Limite persistante des deux moteurs
06	> 60	0,125	Une instance supplémentaire résolue
12	49,840	1,306	Forte réduction du temps de recherche
04	0,070	4,137	Cas défavorable à l'heuristique finale

Temps complets ; valeurs arrondies, avec une seule graine pour DPLL.

Sur `public-04`, DPLL trouve un modèle en 13 décisions, contre 50 088 pour le mode optimisé. Ce contre-exemple montre qu'un branchement défavorable peut annuler le bénéfice d'une propagation plus efficace. Il ne suffit pas à conclure que le hasard est préférable en général.

Inversement, sur `meow_n16_s0`, le mode optimisé prend 1 049 décisions contre 564, mais termine en 0,054 s contre 0,163 s. Le nombre de décisions seul ne prédit donc pas le temps : leur coût et celui des propagations ont changé. Le protocole mesure le gain *combiné* du moteur et de l'heuristique ; il n'isole pas leur contribution respective.

6.4 Validation et portée des conclusions

La validation réalisée compte **54 tests**, dont une comparaison à un oracle exhaustif sur 2 000 petites CNF. Les tests couvrent les cas limites, la propagation, les retours arrière et l'interface. Aucun statut incorrect ni modèle invalide n'a été observé dans la campagne finale. Ces contrôles renforcent la confiance, sans remplacer les invariants de correction.

Les mesures finales utilisent Python 3.12.13 sur macOS 15.7.3 ARM64, avec une seule exécution et une graine DPLL par instance. Une campagne à plusieurs graines et répétitions serait nécessaire pour quantifier la variabilité. Les mesures Q5 et Q6 ne sont pas directement interchangeables : Q5 compare les heuristiques avec du temps CPU et des processus concurrents ; Q6 mesure aussi les lancements complets et utilise d'autres graines.

La recherche reste exponentielle, sans apprentissage de clauses, et H4d parcourt encore la formule à chaque décision. Des tests sur de plus grandes grilles, des UNSAT plus difficiles et la machine de compétition élargiraient la validation. Une comparaison à heuristique identique avant/après changement de propagation isolerait mieux l'origine du gain.

Conclusion. Le projet montre que la performance dépend à la fois de la structure de l'instance, de la polarité choisie et du coût de la propagation. L'optimisation répond aux observations de Q5 et améliore le bilan sur le corpus final, tout en conservant des limites identifiées.

Usage des assistants IA

Nous avons utilisé des assistants IA lorsque nous rencontrions des blocages dans la modélisation, afin d'explorer des pistes et de mieux comprendre les problèmes rencontrés. Ils nous ont également servi de support pour améliorer la structure générale du code et corriger sa syntaxe. Enfin, nous nous en sommes servis pour nous aider à rédiger et à reformuler certaines parties de ce rapport.

Traçabilité et reproduction. Sources : rapports détaillés, `results/q5_runs.csv` et `results/q6_runs.csv`. Les mesures existantes sont réanalysées, sans nouvelle campagne.

Q5 : `uv run python bench_q5.py report -timeout 60`. Q6 : `uv run python bench_q6.py report`.

Résolution : `uv run python sat.py instance.cnf -mode optimized -output-model`.